DEVELOPMENT PACKAGE 1

Platform Foundation

Project: Bidra Platform
Package: 1
Version: 1.0
Purpose: Start development

---

1. Objective

Build the complete technical foundation for Bidra.

This package does **not** implement business features.

The goal is to create a clean, production-ready project skeleton that all future modules can safely build on.

---

2. AI Role

Act as the Lead Principal Software Engineer for Bidra.

You are responsible for creating a maintainable, secure and scalable foundation.

Do not rush.

Do not invent business functionality.

Do not implement Campaigns, Needs, Contributions, Payments, Volunteers, Time Credits, Rewards or Admin dashboards in this package.

---

3. Specifications to Use

Before coding, load and follow:

- Volume 4: System Architecture
- Volume 10: Engineering Standards
- Volume 11: AI Development Instructions
- Volume 12 Part 1: AI Master System Prompt

- Volume 12 Part 2: Project Initialization & Foundation Build Prompt
- Volume 13 Part 1: Master Implementation Roadmap

---

4. Scope

This package includes:

- Monorepo setup
- Frontend app
- Backend API app
- Worker app
- Shared packages
- PostgreSQL
- Prisma
- Redis
- BullMQ
- Docker Compose
- CI foundation
- Environment validation
- Logging
- Error handling
- Health checks
- OpenAPI setup
- Testing setup
- Basic Design System setup
- Authentication infrastructure only

---

5. Required Tech Stack

Frontend

- Next.js App Router
- React
- TypeScript
- Tailwind CSS
- shadcn/ui
- TanStack Query
- React Hook Form
- Zod

Backend

- NestJS

- TypeScript
- Prisma
- PostgreSQL
- Redis
- BullMQ
- OpenAPI / Swagger

Infrastructure

- pnpm workspaces
- Docker
- Docker Compose
- GitHub Actions
- ESLint
- Prettier
- Husky
- lint-staged
- Commitlint

---

6. Required Repository Structure

Create:

```
bidra/
  apps/
    web/
    api/
    worker/
  packages/
    database/
    ui/
    config/
    validation/
    auth/
    types/
    utils/
  docs/
  infrastructure/
    docker/
  scripts/
  .github/
    workflows/
```

Every app and package must compile independently.

---

7. Frontend Foundation

Create `apps/web`.

Required:

- Next.js App Router
- TypeScript
- Tailwind
- shadcn/ui setup
- Global layout
- Home page placeholder
- Error page
- Loading page
- Not found page
- Theme provider
- Query provider
- Toast provider
- Basic navigation shell
- Shared dashboard shell placeholder

Do not create business pages yet.

Allowed pages:

```
/
/login-placeholder
/register-placeholder
/health
```

No real authentication flow yet.

---

8. Backend API Foundation

Create `apps/api`.

Required NestJS modules:

```
health
config
logging
errors
auth
users
audit
queues
storage
search
notifications
```

Important:

- `auth` and `users` should be infrastructure only.
- Do not implement complete registration or login yet.
- Only create structure, interfaces and placeholder services where needed.

Required backend infrastructure:

- Global validation pipe
- Global exception filter
- Request ID middleware
- Structured logger
- CORS config
- Helmet security headers
- OpenAPI setup
- Environment validation
- Graceful shutdown

---

9. Worker Foundation

Create `apps/worker`.

Required:

- BullMQ connection
- Redis connection
- Queue registration
- Worker bootstrap
- Graceful shutdown
- Structured logging

Create queues:

```
email
notifications
stripe
search
qr
reports
cleanup
```

Processors may be empty or placeholder only.

No business jobs yet.

---

10. Database Foundation

Create `packages/database`.

Required:

- Prisma setup
- PostgreSQL connection
- Initial migration
- Prisma client export
- Seed script
- Database health check helper

Foundational models only:

```
User
UserSession
Role
Permission
RolePermission
AuditLog
FeatureFlag
SystemSetting
```

Do not create:

- Organization
- Campaign
- Need
- Contribution
- Payment
- Volunteer
- Time Credit
- Reward

Those belong to later packages.

---

11. Shared Packages

Create:

`packages/config`

- Environment schema
- Config loader
- Typed configuration

`packages/types`

- Shared base types
- API response types

- Pagination types
- Error types

packages/validation

- Shared Zod helpers
- Common schemas

packages/auth

- Permission type definitions
- Auth helper interfaces
- Token interface definitions

packages/ui

- Button
- Input
- Card
- Dialog
- Badge
- Table shell
- Form shell
- Empty state
- Loading state

packages/utils

- Date helpers
- ID helpers
- String helpers
- Logger helpers

No business logic in utilities.

---

12. API Response Standard

Implement standard response helpers:

```
{
  "data": {},
  "meta": {},
  "links": {}
}
```

Implement standard error response:

```
{
  "error": {
```

```
    "code": "ERROR_CODE",
    "message": "Human-readable message.",
    "requestId": "req_123",
    "details": []
  }
}
```

---

13. Health Endpoints

Implement:

```
GET /health
GET /ready
GET /live
```

Health checks must verify:

- API process
- PostgreSQL
- Redis
- Queue connection

---

14. OpenAPI

Set up Swagger/OpenAPI.

Required:

```
/api/docs
/api/docs-json
```

Document:

- Health endpoints
- Error format
- Auth placeholder endpoints if created

---

15. Docker Compose

Create local development environment.

Services:

```
web
```

```
api
worker
postgres
redis
meilisearch
mailpit
```

Requirements:

- Hot reload
- Shared environment variables
- Persistent database volume
- Health checks
- Local network

---

16. CI Pipeline

Create GitHub Actions workflow.

On pull request:

- Install dependencies
- Type check
- Lint
- Run tests
- Build apps

Do not deploy yet.

---

17. Testing Setup

Configure:

- Unit test framework
- Integration test setup
- E2E test placeholder
- Test database config
- Test utilities

Minimum tests:

- API health test
- Config validation test
- Error response test
- Database connection test

- Worker bootstrap test

---

18. Logging

Implement structured logging.

Every API request should include:

- requestId
- method
- path
- statusCode
- duration
- userId if available
- environment

Never log:

- passwords
- tokens
- API keys
- secrets

---

19. Security Foundation

Implement:

- Helmet
- CORS
- Rate limit foundation
- Secure cookie configuration placeholder
- Environment secret validation
- Basic security headers

Do not implement MFA yet.

---

20. Documentation

Create:

`README.md`

```
docs/development-setup.md
docs/architecture-overview.md
docs/environment-variables.md
docs/testing.md
docs/docker.md
docs/api.md
```

Documentation must match actual implementation.

No fake docs.

---

21. Explicit Non-Scope

Do **not** implement:

- Real registration
- Real login
- Organizations
- Campaigns
- Needs
- Contributions
- Stripe payment workflows
- Volunteer opportunities
- Time Credits
- Rewards
- Admin dashboard
- Reporting
- Production deployment

This package is foundation only.

---

22. Acceptance Criteria

Package 1 is complete when:

- Monorepo exists.
- `pnpm install` works.
- `pnpm dev` starts web, API and worker.
- Docker Compose starts all services.
- API health endpoints work.
- PostgreSQL connects.
- Redis connects.
- BullMQ queues initialize.
- Prisma migration runs.
- OpenAPI docs load.

- Frontend loads.
- Shared UI package compiles.
- CI pipeline passes.
- Tests pass.
- No business features are implemented.
- Documentation is accurate.

---

23. AI Execution Checklist

Before finishing, verify:

✓ No business modules created.

✓ No undocumented dependencies added.

✓ No hardcoded secrets.

✓ No skipped validation.

✓ No disabled lint/type checks.

✓ No placeholder TODOs left in critical paths.

✓ Every app compiles.

✓ Every package compiles.

✓ Tests pass.

✓ Docs updated.

---

24. Final Instruction to AI Coding Tool

Build only the foundation.

Do not move ahead into product features.

This package must create the technical base that future packages can rely on.

The output should be clean enough that Development Package 2 can immediately implement Authentication & Identity without restructuring the repository.